

National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

School of Electrical Engineering and Computer Science

CS-250: Data Structure and Algorithms

Class: BSDS 3A

Name	Tayyab Ali Sandhu
CMS ID:	540469
Instructor:	Dr. Fahad Ahmed Satti
Lab Engineer:	Hassan Mohtashim

Lab 6: Stacks and Queues

Date: 12th October 2025

Time: 10:00 am – 12:50 pm



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Github link

https://github.com/tsandhubsds25seecs-code/Lab6_DSA

Lab Tasks

Stacks and Queues

Introduction

In this lab, we will study the implementation of **Stacks** and **Queues**, two fundamental linear data structures used in computer science. A **Stack** follows the *Last In, First Out (LIFO)* principle, where the most recently added element is removed first, while a **Queue** follows the *First In, First Out (FIFO)* principle, where the earliest added element is removed first.

Objectives

Objective of this lab is to enable students to build stack ADT using linked list and arrays, perform the following tasks on it and analyze the performance of each implementation.

Tools/Software Requirement

Visual Studio Code in Unix/Linux Environment.

Task 1: Expression Conversion and Evaluation

Write a program that converts arithmetic expressions between infix, postfix, and prefix forms, and evaluates them. You must implement the stack data structure yourselves (using arrays or linked lists). Use of built-in library is not allowed.

Functions to implement:

1. `string infix_to_postfix(string expr)`
2. `string infix_to_prefix(string expr)`
3. `string postfix_to_infix(string expr)`
4. `double evaluate_postfix(string expr)`

5. `double evaluate_prefix(string expr)`

Solution

Code:

```cpp
#include <iostream>
#include <string>
#include <cmath>
using namespace std;


// ---------------- Character Stack ----------------
class CharStack {
    char arr[200];
    int top;
public:
    CharStack() { top = -1; }
    bool isEmpty() { return top == -1; }
    bool isFull()  { return top == 199; }
    void push(char c) {
        if (isFull()) { cout << "Stack Overflow\n"; return; }
        arr[++top] = c;
    }
    char pop() {
        if (isEmpty()) { cout << "Stack Underflow\n"; return '\0'; }
        return arr[top--];
    }
    char peek() {
        if (isEmpty()) return '\0';
        return arr[top];
    }
};
// ---------------- Number Stack ----------------
class NumStack {
    double arr[200];
```

```
    int top;
public:
    NumStack() { top = -1; }
    bool isEmpty() { return top == -1; }
    void push(double x) { arr[++top] = x; }
    double pop() { return arr[top--]; }
};
// ---------------- String Stack ----------------
class StrStack {
    string arr[200];
    int top;
public:
    StrStack() { top = -1; }
    bool isEmpty() { return top == -1; }
    void push(string s) { arr[++top] = s; }
    string pop() { return arr[top--]; }
};
// ---------------- Helpers ----------------
int priority(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}
bool isOperator(char c) {
    return c=='+' || c=='-' || c=='*' || c=='/' || c=='^';
}
bool isOperand(char c) {
    return (c>='a' && c<='z') || (c>='A' && c<='Z') || (c>='0' && c<='9');
}
// ---------------- 1) Infix -> Postfix ----------------
string infix_to_postfix(string expr) {
    CharStack st;
```

```
    string output = "";
    for (int i = 0; i < (int)expr.size(); i++) {
        char c = expr[i];
        if (c == ' ') continue;
        if (isOperand(c)) {
            output += c;
        }
        else if (c == '(') {
            st.push(c);
        }
        else if (c == ')') {
            while (!st.isEmpty() && st.peek() != '('
                output += st.pop();
            if (!st.isEmpty()) st.pop();   // discard '('
        }
        else if (isOperator(c)) {
            while (!st.isEmpty() && st.peek() != '(' &&
                   priority(st.peek()) >= priority(c))
                output += st.pop();
            st.push(c);
        }
    }
    while (!st.isEmpty()) output += st.pop();
    return output;
}// ---------------- 2) Infix -> Prefix ----------------
string reverse_and_swap(string s) {
    string r = "";
    for (int i = (int)s.size() - 1; i >= 0; i--) {
        char c = s[i];
        if (c == '(') c = ')';
        else if (c == ')') c = '(';
        r += c;
    }
```

```
    return r;

}

string infix_to_prefix(string expr) {

    string rev  = reverse_and_swap(expr);

    string post = infix_to_postfix(rev);

    string pre  = "";

    for (int i = (int)post.size() - 1; i >= 0; i--)

        pre += post[i];

    return pre;

}// ---------------- 3) Postfix -> Infix ----------------

string postfix_to_infix(string expr) {

    StrStack st;

    for (int i = 0; i < (int)expr.size(); i++) {

        char c = expr[i];

        if (c == ' ') continue;

        if (isOperand(c)) {

            st.push(string(1, c));

        } else if (isOperator(c)) {

            string b = st.pop();     // right

            string a = st.pop();     // left

            st.push("(" + a + c + b + ")");

        }

    }

    string r = st.pop();

    if (r.size() >= 2 && r.front() == '(' && r.back() == ')')

        r = r.substr(1, r.size() - 2);

    return r;

}

// ---------------- 4) Evaluate Postfix ----------------

double evaluate_postfix(string expr) {

    NumStack st;

    for (int i = 0; i < (int)expr.size(); i++) {

        char c = expr[i];
```

```
        if (c == ' ') continue;
        if (c >= '0' && c <= '9') {
            st.push(c - '0');
        } else if (isOperator(c)) {
            double b = st.pop();      // RIGHT (first pop)
            double a = st.pop();      // LEFT
            double r = 0;
            if (c == '+') r = a + b;
            if (c == '-') r = a - b;
            if (c == '*') r = a * b;
            if (c == '/') r = a / b;
            if (c == '^') r = pow(a, b);
            st.push(r);
        }
    }
    return st.pop();
}
// ---------------- 5) Evaluate Prefix ----------------
double evaluate_prefix(string expr) {
    NumStack st;
    for (int i = (int)expr.size() - 1; i >= 0; i--) {
        char c = expr[i];
        if (c == ' ') continue;
        if (c >= '0' && c <= '9') {
            st.push(c - '0');
        } else if (isOperator(c)) {
            double a = st.pop();      // LEFT (first pop in reverse scan)
            double b = st.pop();      // RIGHT
            double r = 0;
            if (c == '+') r = a + b;
            if (c == '-') r = a - b;
            if (c == '*') r = a * b;
            if (c == '/') r = a / b;
```

```
            if (c == '^') r = pow(a, b);
            st.push(r);
        }
    }
    return st.pop();
}
// ---------------- Main ----------------
int main() {
    cout << "===== TASK 1: Expression Conversion & Evaluation =====\n\n";
   string infix = "A+B*C/D-F+A^E";
    cout << "Infix   : " << infix << "\n";
    cout << "Postfix : " << infix_to_postfix(infix) << "\n";
    cout << "Prefix  : " << infix_to_prefix(infix) << "\n\n";
    cout << "Postfix 623+-382/+*2^3+ = "
         << evaluate_postfix("623+-382/+*2^3+") << "  (expected 52)\n";
    cout << "Postfix 45+72-*         = "
         << evaluate_postfix("45+72-*") << "  (expected 45)\n";
    cout << "Prefix  +*23/84         = "
         << evaluate_prefix("+*23/84") << "  (expected 8)\n";
    cout << "Postfix->Infix 342*+    = "
         << postfix_to_infix("342*+") << "\n";
    return 0;
}
```

Output:

```
tsandhu_@TAYYAB-ALI-SANDHU:~$ g++ -std=c++17 lab6_task1.cpp -o lab6_t
./lab6_task1
===== TASK 1: Expression Conversion & Evaluation =====

Infix   : A+B*C/D-F+A^E
Postfix : ABC*D/+F-AE^+
Prefix  : +A-*B/CD+F^AE

Postfix 623+-382/+*2^3+ = 52  (expected 52)
Postfix 45+72-*         = 45  (expected 45)
Prefix  +*23/84         = 8  (expected 8)
Postfix->Infix 342*+    = 3+(4*2)
tsandhu_@TAYYAB-ALI-SANDHU:~$
```

Task 2: Expression Validators Using Stack

Implement two stack-based validators to check the correctness of brackets and HTML/XML tag structures.

Part A — Bracket Validator

Write a program to verify if all types of brackets — (), {}, and [] — are balanced and correctly nested.

Function:

```
isValidBrackets(s: str) -> bool
```

Solutions

Part B — HTML/XML Tag Validator

Validate a simplified HTML/XML string for proper nesting and closing of tags using a stack.

Function:

```
validate_markup(s: str) -> (bool, list_of_errors)
```

Solutions

Code :

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;


// ---------------- Character Stack ----------------
class CharStack {
    char arr[200];
    int top;
public:
    CharStack() { top = -1; }
    bool isEmpty() { return top == -1; }
    void push(char c) { arr[++top] = c; }
    char pop() { return arr[top--]; }
};
```

```cpp
// ---------------- String Stack ----------------
class StrStack {
    string arr[200];
    int top;
public:
    StrStack() { top = -1; }
    bool isEmpty() { return top == -1; }
    void push(string s) { arr[++top] = s; }
    string pop() { return arr[top--]; }
};
```

```cpp
// ---------------- Part A: Bracket Validator ----------------
bool isValidBrackets(string s) {
    CharStack st;
    for (int i = 0; i < (int)s.size(); i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
```

```
            st.push(c);
        }
        else if (c == ')' || c == ']' || c == '}') {
            if (st.isEmpty()) return false;
            char open = st.pop();
            if (c == ')' && open != '(') return false;
            if (c == ']' && open != '[') return false;
            if (c == '}' && open != '{') return false;
        }
    }
    return st.isEmpty();
}
```

```
// ---------------- Part B: HTML/XML Validator ----------------
bool validate_markup(string s, vector<string>& errors) {
    StrStack st;
    int i = 0;
    while (i < (int)s.size()) {
        if (s[i] == '<') {
            int j = i + 1;
            while (j < (int)s.size() && s[j] != '>') j++;
            if (j == (int)s.size()) {
                errors.push_back("Missing '>' near position " + to_string(i));
                return false;
            }
            string tag = s.substr(i + 1, j - i - 1);

            if (!tag.empty() && tag[0] == '/') {
                string name = tag.substr(1);
                if (st.isEmpty()) {
                    errors.push_back("No opener for </" + name + ">");
                } else {
                    string open = st.pop();
```

```
                    if (open != name)
                        errors.push_back("<" + open + "> closed by </" + name + ">");
                }
            }
            else if (!tag.empty() && tag[tag.size()-1] == '/') {
                // self-closing <br/> — ignore
            }
            else {
                string name = tag;
                int sp = tag.find(' ');
                if (sp != -1) name = tag.substr(0, sp);
                st.push(name);
            }
            i = j + 1;
        } else {
            i++;
        }
    }
    while (!st.isEmpty())
        errors.push_back("Unclosed <" + st.pop() + ">");
    return errors.empty();
}
// ---------------- Main ----------------
int main() {
    cout << "===== TASK 2: Expression Validators =====\n\n";
    cout << "-- Bracket Validator --\n";
    cout << "{[()]}  -> " << (isValidBrackets("{[()]}") ? "valid" : "invalid") << "\n";
    cout << "{[(])}  -> " << (isValidBrackets("{[(])}") ? "valid" : "invalid") << "\n";
    cout << "(A+B)*C -> " << (isValidBrackets("(A+B)*C") ? "valid" : "invalid") << "\n";
    cout << "A+B)    -> " << (isValidBrackets("A+B)") ? "valid" : "invalid") << "\n\n";
```

```
    cout << "-- HTML/XML Validator --\n";
    vector<string> errs;
```

```
    bool ok = validate_markup("<html><body>Hi</body></html>", errs);
    cout << "Test 1 valid? " << ok << "\n";
```

```
    errs.clear();
    ok = validate_markup("<a><b></a></b>", errs);
    cout << "Test 2 valid? " << ok << "\n";
    for (int i = 0; i < (int)errs.size(); i++)
        cout << "   error: " << errs[i] << "\n";
```

```
    return 0;
}
```

Output:

```
===== TASK 2: Expression Validators =====

-- Bracket Validator --
{[()]}  -> valid
{[])}   -> invalid
(A+B)*C -> valid
A+B)    -> invalid

-- HTML/XML Validator --
Test 1 valid? 1
Test 2 valid? 0
   error: <b> closed by </a>
   error: <a> closed by </b>
tsandhu_@TAYYAB-ALI-SANDHU:~/Lab6_DSA$
```

Task 3: Print Server Request Handling System

Implement a circular queue with dynamic resizing to simulate a Print Server Request Handling System.

Functions to implement:

- `enqueue(x)`
- `dequeue() -> x`
- `front() -> x`
- `isEmpty() -> bool`
- `isFull() -> bool`
- `resize(new_capacity)` (auto-triggered when full or too sparse)

Simulate the scenario where each print request has an ID, arrival time, and processing time. Display the queue state after each enqueue and dequeue operation, ensuring that no request data is lost during resizing.

Test the system by enqueuing and dequeuing multiple requests and calculating the average waiting time for all processed requests.

Code:

```cpp
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;


// ---------------- Print Request ----------------
struct PrintRequest {
    int    id;
    double arrivalTime;
    double processingTime;
};
```

```cpp
// ---------------- Circular Queue (dynamic) ----------------
class CircularQueue {
    PrintRequest* items;
    int cap;
    int front;
    int rear;
```

```
    int Count;
```

```
public:
    CircularQueue(int c = 4) {
        cap   = c;
        items = new PrintRequest[cap];
        front = cap - 1;
        rear  = cap - 1;
        Count = 0;
    }
    ~CircularQueue() { delete[] items; }
```

```
    bool isEmpty() { return Count == 0; }
    bool isFull()  { return Count == cap; }
    int  size()    { return Count; }
    int  capacity(){ return cap; }
```

```
    // ---------- resize (preserves FIFO order) ----------
    void resize(int newCap) {
        PrintRequest* newItems = new PrintRequest[newCap];
        for (int i = 0; i < Count; i++)
            newItems[i] = items[(front + 1 + i) % cap];
        delete[] items;
        items = newItems;
        cap   = newCap;
        front = newCap - 1;
        rear  = Count - 1;
    }
```

```
    void enqueue(PrintRequest r) {
        if (isFull()) {
            cout << "   >> Queue FULL - growing " << cap
```

```
            << " -> " << cap * 2 << "\n";

        resize(cap * 2);

    }

    rear = (rear + 1) % cap;

    items[rear] = r;

    Count++;

}
```

```
PrintRequest dequeue() {

    front = (front + 1) % cap;

    PrintRequest r = items[front];

    Count--;

    if (cap > 4 && Count < cap / 4) {

        cout << "   >> Queue SPARSE - shrinking " << cap

             << " -> " << cap / 2 << "\n";

        resize(cap / 2);

    }

    return r;

}
```

```
PrintRequest getFront() {

    return items[(front + 1) % cap];

}
```

```
void display(string label) {

    cout << label << "  [size=" << Count

         << ", cap=" << cap << "]  ->  ";

    if (isEmpty()) {

        cout << "(empty)";

    } else {

        for (int i = 0; i < Count; i++) {

            PrintRequest r = items[(front + 1 + i) % cap];
```

```
                cout << "R" << r.id
                     << "(a=" << r.arrivalTime
                     << ",p=" << r.processingTime << ") ";
            }
        }
        cout << "\n";
    }
};
// ---------------- Main ----------------
int main() {
    cout << "===== TASK 3: Print Server Simulation =====\n";
    cout << "Commands:\n";
    cout << "  add   - enqueue a new print request\n";
    cout << "  serve - dequeue the front request\n";
    cout << "  show  - display the current queue\n";
    cout << "  quit  - end and show summary\n\n";
```

```
    CircularQueue q(4);
    int    nextId      = 1;
    double totalWait   = 0;
    int    servedCount = 0;
    string cmd;
    while (true) {
        cout << "\n> ";
        cin >> cmd;
```

```
        // ------ QUIT ------
        if (cmd == "quit") break;
        // ------ ADD ------
        if (cmd == "add") {
            double arr, proc;
            cout << "   Enter arrival time: ";
            cin >> arr;
```

```
        cout << "   Enter processing time: ";
        cin >> proc;
        PrintRequest r;
        r.id             = nextId++;
        r.arrivalTime    = arr;
        r.processingTime = proc;
        q.enqueue(r);
        q.display("Enqueue R" + to_string(r.id));
    }
    // ------ SERVE ------
    else if (cmd == "serve") {
        if (q.isEmpty()) {
            cout << "   Queue is empty - nothing to serve.\n";
            continue;
        }
        PrintRequest r = q.dequeue();
        totalWait += r.arrivalTime;    // simplified wait
        servedCount++;
        q.display("Dequeue R" + to_string(r.id) +
                  " (arrived@" + to_string((int)r.arrivalTime) + ")");
    }
    // ------ SHOW ------
    else if (cmd == "show") {
        q.display("Current queue");
        if (!q.isEmpty()) {
            PrintRequest f = q.getFront();
            cout << "   front is R" << f.id
                 << " (arrived@" << f.arrivalTime << ")\n";
        }
    }
    // ------ UNKNOWN ------
    else {
        cout << "   Unknown command. Try: add / serve / show / quit\n";
```

```
        }
    }
    // ------ Summary ------
    cout << "\n----- Summary -----\n";
    cout << "Total served      : " << servedCount << "\n";
    cout << "Total waiting time: " << totalWait << "\n";
    cout << fixed << setprecision(2);
    if (servedCount > 0)
        cout << "Average waiting   : "
             << totalWait / servedCount << " time units\n";
    else
        cout << "No requests were served.\n";
    return 0;
}
```

Output:

```
tsandhu_@TAYYAB-ALI-SANDHU:~/Lab6_DSA$ cd ~/Lab6_DSA
g++ -std=c++17 task3.cpp -o task3
./task3
===== TASK 3: Print Server Simulation =====
Commands:
  add   - enqueue a new print request
  serve - dequeue the front request
  show  - display the current queue
  quit  - end and show summary


> add
   Enter arrival time: 0
   Enter processing time: 3
Enqueue R1  [size=1, cap=4]  ->  R1(a=0,p=3)

> serve
Dequeue R1 (arrived@0)  [size=0, cap=4]  ->  (empty)

> add
   Enter arrival time: 1
   Enter processing time: 2
Enqueue R2  [size=1, cap=4]  ->  R2(a=1,p=2)

> 3
   Unknown command. Try: add / serve / show / quit

> add
   Enter arrival time: 3
   Enter processing time: 4
Enqueue R3  [size=2, cap=4]  ->  R2(a=1,p=2) R3(a=3,p=4)

> add
   Enter arrival time: 5
```

```
   Enter processing time: 7
Enqueue R4  [size=3, cap=4]  ->  R2(a=1,p=2) R3(a=3,p=4) R4(a=5,p=7)

> serve
Dequeue R2 (arrived@1)  [size=2, cap=4]  ->  R3(a=3,p=4) R4(a=5,p=7)

> show
Current queue  [size=2, cap=4]  ->  R3(a=3,p=4) R4(a=5,p=7)
   front is R3 (arrived@3)

> quit

----- Summary -----
Total served      : 2
Total waiting time: 1
Average waiting   : 0.50 time units
tsandhu_@TAYYAB-ALI-SANDHU:~/Lab6_DSA$
```

Task 4: Waiting Room Management System

You are required to implement a Waiting Room Management System for an emergency ward using an array-based queue.

Scenario:
Patients arrive at the hospital and are registered in a first-come, first-served manner. Each patient is assigned a unique auto-generated ID (starting from 1). Patients are served according to their registration order.

Functional Requirements:

Your program will contain the following methods:

RegisterPatient(): This method assigns an Id (which is auto generated) to a patient and register him/her to the system.

ServePatient(): This method calls a patient to provide hospital service to him/her.

CancelAll(): This method cancels all appointments of the patients so that the doctor can go to lunch.

CanDoctorGoHome(): This method returns true if no one is waiting, otherwise, returns false.

ShowAllPatient(): This method shows all ids of the waiting patients in SORTED order.

Code :

```cpp
#include <iostream>

using namespace std;



class WaitingRoom {

    int* q;          // dynamic array of patient IDs

    int  cap;        // current capacity

    int  head;       // index of the front patient

    int  Count;      // number of patients currently waiting

    int  nextId;     // auto-incrementing ID counter
```

```cpp
public:

    // ---------- Constructor ----------

    WaitingRoom(int c = 4) {

        cap    = c;

        q      = new int[cap];

        head   = 0;

        Count  = 0;

        nextId = 1;

    }
```

```cpp
    // ---------- Destructor ----------

    ~WaitingRoom() {

        delete[] q;

    }
```

```cpp
    // ---------- RegisterPatient ----------

    void RegisterPatient() {

        // grow if full

        if (Count == cap) {

            int newCap = cap * 2;
```

```
        int* nq = new int[newCap];
        for (int i = 0; i < Count; i++)
            nq[i] = q[(head + i) % cap];
        delete[] q;
        q     = nq;
        cap   = newCap;
        head  = 0;
        cout << "   (queue was full - grown to " << cap << ")\n";
    }
```

```
    int tail = (head + Count) % cap;
    q[tail]  = nextId;
    cout << "Registered patient with ID = " << nextId << "\n";
    nextId++;
    Count++;
}
```

```
// ---------- ServePatient ----------
void ServePatient() {
    if (Count == 0) {
        cout << "No patient waiting - nothing to serve.\n";
        return;
    }
    int id = q[head];
    head = (head + 1) % cap;
    Count--;
    cout << "Serving patient with ID = " << id << "\n";
}
```

```
// ---------- CancelAll ----------
void CancelAll() {
    head  = 0;
```

```
    Count = 0;
    cout << "All appointments cancelled.\n";
}
```

```
// ---------- CanDoctorGoHome ----------
bool CanDoctorGoHome() {
    return Count == 0;
}
```

```
// ---------- ShowAllPatient (sorted) ----------
void ShowAllPatient() {
    if (Count == 0) {
        cout << "No patients waiting.\n";
        return;
    }
```

```
    // Copy IDs into a temporary array
    int* ids = new int[Count];
    for (int i = 0; i < Count; i++)
        ids[i] = q[(head + i) % cap];
```

```
    // Bubble sort (simple and beginner-friendly)
    for (int i = 0; i < Count - 1; i++)
        for (int j = 0; j < Count - 1 - i; j++)
            if (ids[j] > ids[j + 1]) {
                int t     = ids[j];
                ids[j]    = ids[j + 1];
                ids[j + 1] = t;
            }
```

```
    cout << "Waiting patients (sorted): ";
    for (int i = 0; i < Count; i++)
```

```
            cout << ids[i] << " ";
        cout << "\n";
```

```
        delete[] ids;
    }
```

```
    // ---------- Helper: how many are waiting ----------
    int size() { return Count; }
};
```

```
// ---------------- Main ----------------
int main() {
    cout << "===== TASK 4: Emergency Ward Waiting Room =====\n";
    cout << "Commands:\n";
    cout << "  reg    - register a new patient\n";
    cout << "  serve  - serve the front patient\n";
    cout << "  cancel - cancel all appointments\n";
    cout << "  home   - check if doctor can go home\n";
    cout << "  show   - show all waiting patients (sorted)\n";
    cout << "  quit   - exit\n\n";
```

```
    WaitingRoom w(4);       // start with capacity 4
```

```
    string cmd;
    while (true) {
        cout << "\n> ";
        cin >> cmd;
```

```
        if (cmd == "quit") {
            break;
        }
```

```
        else if (cmd == "reg") {
            w.RegisterPatient();
        }
        else if (cmd == "serve") {
            w.ServePatient();
        }
        else if (cmd == "cancel") {
            w.CancelAll();
        }
        else if (cmd == "home") {
            if (w.CanDoctorGoHome())
                cout << "Yes - doctor can go home.\n";
            else
                cout << "No - " << w.size() << " patient(s) still waiting.\n";
        }
        else if (cmd == "show") {
            w.ShowAllPatient();
        }
        else {
            cout << "Unknown command. Try: reg / serve / cancel / home / show / quit\n";
        }
    }

    cout << "\nExiting waiting room system.\n";
    return 0;
}
```

Output:

```
tsandhu_@TAYYAB-ALI-SANDHU:~$ cd ~/Lab6_DSA
g++ -std=c++17 task4.cpp -o task4
./task4
===== TASK 4: Emergency Ward Waiting Room =====
Commands:
  reg    - register a new patient
  serve  - serve the front patient
  cancel - cancel all appointments
  home   - check if doctor can go home
  show   - show all waiting patients (sorted)
  quit   - exit


> reg
Registered patient with ID = 1

> tayyab
Unknown command. Try: reg / serve / cancel / home / show / quit

> reg
Registered patient with ID = 2

> reg
Registered patient with ID = 3

> reg
Registered patient with ID = 4

> reg
   (queue was full - grown to 8)
Registered patient with ID = 5

> serve
Serving patient with ID = 1
```

```
> show
Waiting patients (sorted): 2 3 4 5

> home
No - 4 patient(s) still waiting.

> cancel
All appointments cancelled.

> home
Yes - doctor can go home.

> quit

Exiting waiting room system.
tsandhu_@TAYYAB-ALI-SANDHU:~/Lab6_DSA$
```